2027年热门会议推荐

具体咨询:白老师 13516196409

智驱产品技术领航

AI 时代产品和软件开发前沿系列峰会

核心定位

锚定AI 技术革新前沿,聚焦大模型与智能体工程化落地,解构 AI 时代软件工程架构革新路径,打通 AI 产品从技术突破到产品商业化验证的全链路。

关键词

- 大模型驱动
- 技术趋势
- Agent工程化
- 架构革新
- AI产品0-1
- 商业化验证

4月 PM×AI 产品力领航者大会

2027年4月23-24·杭州

详情查看官网
演讲申报 | 参会购票

6月 Agentic AICon 智能体应用与架构工程大会

2027年6月18-19·上海

详情查看官网
演讲申报 | 参会购票

11月 Agentic AICon 智能体应用与架构工程大会

2027年11月5-6日·深圳

AI 赋能提质增效

QECon全球软件质量&效能大会

核心定位

聚焦 AI 技术在软件研发全流程的落地应用,以“质量保障&效能提升”为双驱动,助力研发团队实现软件研发的智能化转型。

关键词

- AI 赋能研发
- 效能提升
- 测试驱动
- 平台工具链
- 质量保障
- .....

5月 QECon 全球软件质量&效能大会

2027年5月21-22·深圳

9月 QECon 全球软件质量&效能大会

2027年9月10-11日·上海

10月 QECon 全球软件质量&效能大会

2027年10月15-16日·北京

演讲申报 | 参会购票
详情查看官网

生产级Agent设计蓝皮书·行业案例卷

"蓝皮书共建计划"启动——行业与场景专家 招募中

白皮书定义模式，蓝皮书记录落地。
把你的生产案例，写进行业参照系。
你的案例，很可能是这个行业第一份模式应用实践指南。

交流地点：《艾普·AI生产力研究院》展位咨询

扫码参与



共建计划

以真实案例为基础，持续组织行业与场景的 Agent 设计模式应用撰写。蓝皮书不写概念，只记录具名系统——它面对什么约束、踩过什么失败、为什么这样选架构、哪些经验可以迁移到你的行业。

我们招募这样的行业与场景专家

主导或深度参与过 Agent 系统建设，有真实跑在生产环境的案例；
在强监管、高可靠或大规模部署约束下做过工程取舍（金融、制造、政务、医疗、能源、电商等皆宜）；
愿意把一段经验，沉淀成可评审、可迁移的模式案例。

参与共建，你将获得

署名蓝皮书案例贡献者，成果进入 ADPS 公共技术资料长期存档；
用白皮书的模式语言系统梳理自己的实践，让团队经验成为行业参照；
参与共同体模块研讨与模式评审，共同界定下一版模式的边界；
与已在共同体的腾讯、蚂蚁、百度、京东等一线专家跨行业互审。

《企业级 Agent 设计白皮书》发布

纸质版 & 电子版 同步上线

领取地点:《艾普·AI生产力研究院》展位

Agent 正在从「会回答问题的模型」变成「能参与业务流程的系统」。模型仍是引擎,但决定它在生产中稳不稳的,是周围这套工程结构。本白皮书给出智能体工程的可复用架构词汇与评审标准。



27	7×6	3	5	6
矩阵模式规范	认知功能 × 执行拓扑	横切工程面 X1–X3	工程契约	步选型法

智能体工程的「GoF设计模式」

27个可复用Agent设计模式双轴框架

企业级智能体架构评审与选型实战手册

双轴框架

纵轴 · 认知功能

Agent 在做什么类型的工作

感知 Perception | 记忆 Memory | 推理 Reasoning | 行动 Action | 反思 Reflection | 协作 Collaboration | 治理 Governance

横轴 · 执行拓扑

数据与控制怎么流动

链式 · 顺序 | 路由 · 分流 | 并行 · 并发 | 编排 · 协调 | 循环 · 迭代 | 层级 · 分层

获取白皮书电子版

艾普·AI生产力研究院

Artificial Intelligence Institute of Productivity

Artificial Intelligence Institute of Productivity · 中立 · 开放 · 共创

面向 AI 原生软件工程的开放研究与实践组织

驾驭智能·定义 AI 时代新质生产力

深度承接《"人工智能+软件"专项行动实施方案》的专业化研究与交流平台

软件质量与效能 | **智能体工程与应用** | **六大研究方向** | **五大核心模块**

艾普 · AI 生产力研究院（AIIP）是一个中立、开放、共创的专业化研究与交流平台。研究院依托在软件质量与效能、智能体工程与应用等前沿领域的长期实践积累与前瞻研究底蕴，汇聚资深技术专家与企业研发团队等各方力量，共同推进行业能力体系建设、参考标准研究与评估框架研制，致力于成为支撑智能软件产业新质生产力发展的专业化交流平台。

更多详情请查看官网：https://aiip.org.cn/

关注官网

申请加入

与先行者同行，共赴 AI 生产力浪潮

六大研究方向

AI for SE

智能软件工程

AI赋能软件开发全流程、AI Coding 生产力与质量保障、智能测试工程，对标智能编程工具链与智能化技改。

度量体系

质效度量与治理

AI 质效度量模型、Token 成本治理、ROI 分析框架，以及成熟度评估参考框架的研究与共建。

ADPS

Agent 设计模式

Agentic 设计模式、多智能体架构、AI 系统技术栈与最佳实践沉淀，对标技能包与应用商店。

FDE

前线部署工程师

工程实践方法论、行业场景适配、企业落地案例与效能提升路径，对标就业友好与重点行业落地。

SE for AI

AI 工程化落地

AI 系统全生命周期工程化方法，可靠性与质量保障，以及 MLOps / LLMOps 交付与治理框架。

AI for X

AI 生产力

AI深度赋能千行百业，从软件研发走向办公、运营、数据、科研等真实业务价值场景，把工程能力转化为可度量的生产力。

AI 质效宣言

发起单位：QECon组委会 QECon Organizing Committee

从共识到行动，让宣言融入每一次研发实践

回归研发本质 **重塑人机协作** **推动行业共识**

协作共生 高于 局部取代
长效质量 高于 短期提效
适应进化 高于 固化流程

价值创造 高于 技术崇拜
数据驱动 高于 经验依赖
开放共享 高于 封闭自建

加入共建计划
获取电子画面

两层能力递进 · 一层工程底座

AI 增强软件研发全景架构

AI 嵌入 SDLC · 效能提升

• 人主导 • AI 协同

需求	设计	开发	测试	CI / CD	运维 / 运营
行业调研与趋势分析	代码理解与逻辑探测	代码生成与缺陷解决	测试用例与数据生成	构建与部署脚本生成	告警聚合与摘要生成
创意启发与功能探索	技术选型与架构设计	氛围编程与规约编程	测试脚本修复与自愈	流水线失败诊断与修复	故障分析与定位
需求分析与规格细化	接口设计与数据模型生成	单元测试生成与执行	端到端测试生成与维护	无人值守部署与发版	故障处理与系统自愈
需求检查与内容评审	技术方案评审与风险分析	代码评审与问题修复	缺陷发现与根因分析	依赖升级与漏洞修复	用户体验与反馈洞察

AI 研发范式变革 • 效能跃迁

• AI 主导 • 人监督

工程实践
- 驾驭工程（模型决定上限，Harness 提升底线）
- 知识工程（跨仓库代码与文档索引 · 沉淀复用）
- 验证闭环（生成 → 验证 → 修正）
- 质量门禁（构建多层次质量防护网）

流程重构
- 意图驱动、面向结果的交付（替代多阶段接力）
- 对齐在前、执行在后（人谈清边界，AI 自主实现）
- 需求即规格、证据即信任（规格成为人机契约）
- 快速原型、持续迭代（沿用敏捷 · 精益研发思想）

组织重塑
- 交付层抹平前后端分工（跨栈交付 · 功能负责人）
- 架构师专注标准与门禁（定标准 · 守门禁）
- 设计师与产品自主交付（部分需求不经研发）
- 先锋队试点后规模复制（先跑通再逐步推广）

效能度量
- 结果面：多 · 快 · 好 · 省
- 过程面：渗透率 · 成熟度 · 使用效果
- 数据源：代码 · 会话 · 配置 · 需求 · 监控
- 北极星：需求交付周期 · 单位 token 有效价值

AI 工程底座 • SE4AI（Software Engineering for AI）

• 自下而上 • 决定能力上限

模型层
- 多模型接入与路由（模型网关 · 按能力与成本择优）
- 能力分级与降级（强弱模型分级 · 故障回退）
- 领域微调与私有化（微调训练 · 内网部署）
- 模型评测与选型（基准跑分 · 场景实测对比）

Agent 层
- Agent 架构与编排（单/多 Agent · 规划与控制流）
- 上下文工程与记忆（上下文组装压缩 · 长短记忆）
- 运行时沙盒与工具接入（隔离执行 · MCP 与技能调用）
- 灰度发布与回滚（能力评估达标才放量 · 版本化 · 自动回退）

生态层
- 技能与智能体市场（技能复用 · Agent 共享流通）
- MCP Hub 与工具生态（工具目录 · 权限与调用治理）
- 知识资产与语料沉淀（业务知识 · 代码知识 · 过程知识）
- 检索服务与知识图谱（语义检索 · 图谱关系推理）

贯穿三层 | **安全与治理** 数据脱敏 · 提示注入防护 · 最小授权 · 输出合规 | **可观测性** 决策链路追踪 · 上下文溯源 · 失败归因 | **能力评估** 任务成功率 · 人工接管率 · 单任务成本

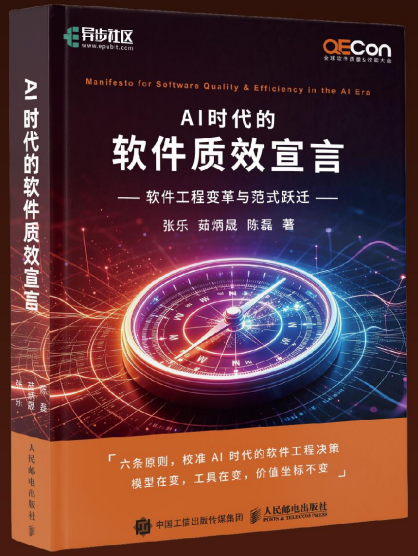
为每一位实践者提供一组稳定的价值坐标——当工具在变、模型在变、最佳实践在变的时候，这些原则帮你判断什么是不变的、什么是值得坚守的。

AI 时代的软件质效宣言

新书发布

— 软件工程变革与范式跃进 —

扫码订阅



系列电子书结构·三篇十九章 (道 · 术 · 势 —— 一条完整的 AI 研发变革路径)

道

第一篇 · 解读篇 (1-8 章)

确立价值观与原则：理解 AI 时代软件质效的本质，重建研发协作的信念底座，是全书的「道」。

术

即将发布

第二篇 · 单点提效篇 (9-14 章)

覆盖需求分析 → 架构设计 → 编码实现 → 测试验证 → CI/CD → 运维运营的关键环节，打通 AI 赋能研发全链路，是全书的「术」。

势

即将发布

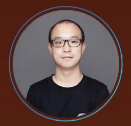
第三篇 · 范式篇 (15-19 章)

当 AI 贯穿整条研发链路之后，组织形态本身将如何变革？从团队结构到协作范式，预见未来，是全书的「势」。

—— 主编 ——



张乐
腾讯
智能化软件工程
资深技术专家

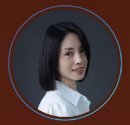


茹炳晟
腾讯研究院
高级技术顾问



陈磊
前京东测试架构师

—— 作者 ——



王玉霞
北京兴云数科
过程改进专家



李佳奇
去哪儿旅行
技术总监



牛万鹏
百度
Comate研发经理



张晔
腾讯端服务质效
技术负责人



张涛
快手质量通道委员



付求爱
华为云
AIOps技术负责人



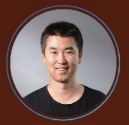
李萌
珊瑚智成
创始人&CEO



张刚
上海英慕科技
资深技术专家



张兆鑫
OPPO AI架构师



王磊
华为 AI技术专家

主办方：智盟创课

智盟创课（北京）科技有限公司（简称“智盟 Stu.”），作为专业IT产研培训服务供应商与AI时代数字化人才发展伙伴，为科技研发型企业及各类组织提供系统性、分层次、重实战的人才培养解决方案，同时搭建行业前沿技术交流平台，致力于成为企业数字化转型进程中人才培养与业务创新的核心支撑力量，推动组织实现成功转型及经营业绩持续增长。

同时，智盟发起并组织三大 IT 领域品牌会议：QECon 软件质量效能大会、SECon+AgentX 智能时代软件工程技术大会、PMxAI 产品力领航者大会、Agentic AICon 智能体应用与架构工程大会，助力产研中心深化数字化转型、聚焦业务价值达成，在产品研发的快速交付与高质量发展之间找到平衡，打通企业全维度学习视角，最终实现人才能力提升与企业竞争优势增强的双重目标。



详情咨询: 010-65798049

多元化学习形式



核心培训方向



软件研发赋能

大模型前沿	架构设计	数据库	产品经理	持续运维	质量管理
移动开发	组织管理	云计算	开发编程	敏捷开发	大数据

Agentic AICon

打破尽力而为：面向智能体互联网（A2A）的跨域可信执行基础设施

范海燕博士 | 万联易达·产业智能体互联网首席架构师



范海燕博士

万联易达 · 产业智能体互联网首席架构师

毕业于国防科技大学，获信息与通信工程博士学位，曾获得全国大学生数学竞赛全国第二名，曾任商汤科技顾问研究员，中国科学院电子学研究所与济钢集团共建企业高级研究员，头部CEX交易所研发部技术负责人等职。具有深厚的分布式系统与密码学优化背景，曾带领团队荣获全球密码学竞赛ZPrize最高奖金。同时拥有负责大型AI项目的丰富经验，曾主导1.9亿资金规模的低空监视服务网以及2亿资金规模的Geo.AI平台的国家级重大项目产业转化，并开创性地将大模型应用于非合作电磁信号分析。现任工业AI架构师，聚焦于跨组织Agent协作架构设计，致力于解决工业级AI应用中的信任、语义与可靠性难题。



范海燕 博士

万联易达 · 产业智能体互联网 · 首席架构师

工业 AI 架构师 · 大模型对齐与多 Agent 协同专家

Agentic AICon

毕业于国防科技大学，获信息与通信工程博士学位，**曾获得全国大学生数学竞赛全国第二名**。曾任**商汤科技顾问研究员、中国科学院电子学研究所与济钢集团共建企业高级研究员、头部CEX交易所研发部技术负责人**等职。

具有深厚的分布式系统与密码学优化背景，曾带领团队荣获**全球密码学竞赛ZPrize最高奖金**。**同时拥有负责大型AI项目的丰富经验，曾主导1.9亿资金规模的低空监视服务网以及2亿资金规模的Geo.AI平台的国家级重大项目产业转化，并开创性地将大模型应用于非合作电磁信号分析。**

现任**万联易达产业智能体互联网首席架构师**，主导"产业智能体互联网"架构设计。该概念于2026 WAIC全球亮相，致力于构建无界、无休、自生长的产业智能体协作网络。

万联易达核心生态

- **产业智能体互联网**
  2026 WAIC全球亮相，重构产业组织形态
- **万联大模型**
  产业级大语言模型，支撑企业级智能决策
- **企业经纪人**
  自动化跨组织业务协作与资源调度
- **CNNIC 战略合作**
  共建产业互联网信任基础设施

ZPrize冠军	数学竞赛全国第二	3.9亿	电磁信号AI开创	2026
		国家级项目		WAIC全球亮相

目录

01 背景与挑战

02 整体解决方案

03 落地实现细节

04 工程案例与收益

05 复盘思考与展望

06 Takeaway总结

PART ONE

背景与挑战

01

跨组织协作的4层难题与结算场景

⚠ 跨组织协作的 4 层难题

当 AI Agent 从「单组织内部」走向「跨组织协作」，团队会遇到四层核心挑战：

跨组织信任

默认不信任，无统一权限模型。不同组织的 Agent 凭什么相信对方身份与能力声明?

语义不一致

同一个词含义不同。采购方的“结算金额”与供应商的理解可能差 **13 万元**。

副作用不可控

付款、发货等动作不可逆。若无确定性保障，Agent 可能执行错误的金额或操作。

可靠性无保障

消息会丢、会重、会乱序。网络故障时流程能否从断点恢复，不重复执行?

场景切入：跨组织结算的口径差异与执行风险

Agentic AICon

7个关键节点的业务冲突与无解决方案的后果

采购方财务 Agent 依据合同签订日汇率，计算应付金额 **500万元**；
供应商财务 Agent 依据到账日汇率，计算应收金额 **487万元**。
双方口径差异 **13万元**。

节点	业务冲突	无解决方案的后果	架构应对
1 材料提交	订单号、合同版本、发票来自三个系统	材料无法核对	contracts/ + validate_receipt
2 任务交接	采购方需将结算任务交接给供应商	交接丢失、任务重复	discovery/ + handoff/ + trust/
3 口径差异	"结算金额"定义不一致	差异 13 万元	semantic/ + Context Agent
4 口径确认	双方需确认采用的规则版本	版本不一致，无法追溯	semantic/ 版本校验
5 生成草案	金额计算依赖口径（签订日or到账日汇率）	口径不一致 → 差13万元	分层披露 + calculate_amount
6 付款审批	大额付款需人工审批	越权执行，责任不可追溯	interrupt() + OPA治理门
7 执行回执	付款发出，需确认对方到账	重复付款，资金无法追回	幂等控制 + verify_payment

Agentic AICon

PART TWO

整体解决方案

02

LangGraph + 10模块的编排内核

跨组织 Agent 协作基础设施架构

跨组织结算任务 (Cross-Organizational Settlement Task)

↓

跨组织基础设施层 (Infrastructure Layer)

五项能力：身份信任 | 工作交接 | 语义一致 | 策略门控 | 可信执行

对齐组 (Alignment)

消除: 理解失败

- **contracts**
  签署语义契约，固定口径与责任边界
- **discovery**
  基于Agent Card的可校验能力发现
- **semantic**
  字段级口径对齐，确保起算点一致
- **context**
  上下文双重治理：语义转换与最小化脱敏
- **models**
  声明模型与判定规则，确保标准一致

交互组 (Interaction)

消除: 传输失败

- **trust**
  四层信任机制(白名单/mTLS/签名/等级)，身份互认与授权依据
- **handoff**
  标准化交接：状态快照、双方确认、责任转移、精确恢复
- **reliability**
  幂等控制(重复请求只执行一次)、重试与断点恢复

保障组 (Assurance)

消除: 执行失败

- **governance**
  策略门控(OPA默认拒绝)：核验身份、比对授权路径、升级审批
- **tools**
  确定性执行：资金动作由纯Python完成，模型输出不触发执行

🔗 TraceContract 证据链 (Audit Chain) —— 贯穿三道防线，聚合于同一 trace_id，消除静默失败

↓

单组织编排层 (Single-Org Orchestration Layer)

LangGraph | AutoGen | Claude Code | ...

架构内核：LangGraph + 10个模块

Agentic AICon

10个模块按职责分为三组，是纵深防御关系

LangGraph 内核 + 10个模块

对齐组

解决"说得通"：双方对任务、口径、委派理解一致

- contracts
- discovery
- semantic
- context
- models

让**错误不发生**

交互组

解决"连得上"：信息与动作跨组织传输时可信、完整

- trust
- handoff
- reliability

让**传输不出错**

保障组

解决"不出事"：权限门 + 确定性执行，构成最后防线

- governance
- tools

让**错误兜得住**

内部多Agent

按需组合，非全量部署

core + discovery + models

跨组织结算

core + semantic + trust + handoff + governance + tools

数据协作 (无资金)

省去 governance & tools, 保留审计校验

图节点与场景角色对应

Agentic AICon

编排内核支持动态路由、中断、恢复、证据累加

节点映射表

图节点	场景角色	对应场景
procurement	采购方Agent	节点1–2
supplier	供应商Agent	节点2
semantic_check	语义契约校验	节点3–4
finance	财务执行	节点5
human_review	人工审批：大额付款	节点6
audit	出证归档 / 驳回	节点7
reject	异常处理：驳回归档	异常路径

build_graph.py

简化版

```
# 构建跨组织结算的图——编排的核心 def build_graph(checkpointer): builder = StateGraph(SettlementState) # 1. 添加核心节点（名称必须与表格一致） builder.add_node("procurement", procurement_agent) builder.add_node("supplier", supplier_agent) builder.add_node("semantic_check", semantic_check_agent) # 语义契约校验 builder.add_node("finance", finance_agent) builder.add_node("human_review", human_review) # 审批门 builder.add_node("audit", audit_agent) builder.add_node("reject", reject_node) # 2. 设置动态路由条件 builder.add_conditional_edges( "semantic_check", route_after_semantic_check, {"proceed": "finance", "reject": "reject"}, ) # 3. human_review 使用 Command(goto=...) 动态路由 # 审批期间流程持久化，断电重启可恢复 return builder.compile(checkpointer=checkpointer)
```

⚡ 动态路由与中断恢复

Agentic AICon

大额付款的审批门：interrupt() 暂停并持久化，断电重启可恢复

审批流程

1. **触发审批**
   500万大额付款，流程在 human_review 节点 interrupt()
2. **持久化状态**
   Checkpointer 保存当前状态，流程不在内存里硬等
3. **人工恢复**
   审批人通过 Command(resume=...) 恢复流程
4. **路由决策**
   批准 → finance_agent；驳回 → reject

💡 **关键：**审批期间流程不在内存里硬等——断电、重启、隔天再批，都能从断点恢复。

human_review.py

● 简化版

```
# 人工审批节点：大额付款的审批门 def human_review(state) -> Command[Literal["finance", "reject"]]: """在节点6处暂停，等待人工审批""" # 1. 中断流程，返回审批请求 approval = interrupt({ "question": "审批付款？ ", "event_id": state["event_id"], "amount": state["amount"] }) # 2. 审批人通过 Command(resume=...) 恢复流程 # interrupt() 返回审批结果 if approval == "approved": return Command( update={"approval_status": "approved"}, goto="finance" # 执行付款 ) return Command( update={"approval_status": "rejected"}, goto="reject" # 驳回归档 ) # 场景对应：节点6 付款审批 # 这段代码就是500万大额付款的实现 # 流程在此暂停并持久化，审批人可隔天再批
```

业务逻辑与节点分离

Agentic AICon

节点是"指挥"——工具是"士兵"，核心原则：LLM只做理解和判断，副作用动作全部是确定性代码

对应模块：tools/

core/nodes/ · 编排者

决定调用什么、传什么、交给谁，依赖LangGraph、State、tools

维度	core/nodes/
定位	编排
职责	决定调用什么、传什么、交给谁
依赖	LangGraph、State、tools
可测	需要mock框架

核心原则：LLM在这个系统里只做两件事：
1. 理解（口径核对）
2. 判断（草稿审核）
金额计算、付款执行这类"错一次就出事"的动作，全部是确定性代码。

tools/ · 执行者

真正干活，纯Python实现，不依赖框架，可独立测试

维度	tools/
定位	执行
职责	真正干活
依赖	纯Python——不依赖框架
可测	可独立测试

validate_receipt.py	校验收货回单格式与签名
calculate_amount.py	按口径契约计算金额
execute_payment.py	调银行接口执行付款
verify_payment.py	确认对方到账回执

模型分层原则

Agentic AICon

不同环节用不同的模型，成本–风险匹配原则：风险越高，越不用概率模型

对应模块：models/

口径快查

低风险

用于快速查询和简单校验

小模型（低延迟、低成本）

- 口径定义快速匹配
- 简单格式校验
- 高频调用场景
- 容错性高

审批辅助

中风险

用于复杂判断和语义理解

大模型（复杂判断）

- 口径差异语义分析
- 草稿内容审核
- 异常情况判断
- 人工审批辅助决策

金额计算 · 付款执行

禁用LLM

确定性代码，见 tools/ 目录，这是把"副作用不可控"从架构层消除的第一道闸

纯Python确定性代码

⚠ 核心原则：风险越高，越不用概率模型。金额计算、付款执行这类"错一次就出事"的动作，全部是确定性代码。

上下文治理与管理

两层分工：治理管"语义怎么来"，管理管"语义怎么用"

对应模块：context/ + semantic/ + contracts/

场景里的问题

场景表节点3、4——口径确认时，双方 Agent 要核对"结算金额"的定义。这份定义从哪来？靠双方业务人员手写维护？滞后且不一致；靠 LLM 直接读合同？**30页合同 + 20张发票，上下文窗口塞不下。**

上下文治理

管"语义怎么来"——确保口径定义的一致性与可追溯性

- **契约发布流程**：definition 和 relations 由治理流程统一发布
- **版本管理**：valid_from 生效日期，version 版本号
- **血缘追溯**：lineage 字段追溯数据来源
- **权限控制**：permission 字段控制谁能看
- **防篡改**：hash 字段覆盖整个契约内容

核心：治理确保"口径定义"是权威的、版本化的、不可篡改的。

上下文管理

管"语义怎么用"——在运行时如何高效加载与使用上下文

- **分层存储**：大文件存S3，小文件存Redis
- **引用传递**：HandoffPacket只传哈希ID + 摘要
- **按需加载**：LLM调用fetch_artifact获取完整内容
- **窗口管理**：30页合同不塞爆prompt，按需获取
- **上下文裁剪**：只加载与当前节点相关的口径

核心：管理确保"口径定义"在运行时高效加载，不塞爆上下文窗口。

上下文治理流程：Context Agent & 置信度

Agentic AICon

Context Agent 自动生成语义契约，置信度评估实现人机高效协同

对应模块：context/

⚡ Context Agent 生成流程

1. **提取定义**
   从合同、发票等原始文档中提取"结算金额"等术语的初始定义
   `extract_definition(documents)`

2. **推断血缘**
   追溯数据来源，构建 lineage 字段，确保可追溯性
   `infer_lineage(sources)`

3. **构建本体**
   构建 Ontology 推理层，包含 class、relations、constraints
   `build_ontology(extracted_data)`

4. **置信度评估**
   对生成的契约草稿进行多维度评分，决定是否需要人工审核
   `assess_confidence(contract_draft)`

📊 置信度评估与路由

四维度加权评分公式：

```
confidence =
  definition_clarity × 0.4 +
  lineage_completeness × 0.2 +
  ontology_consistency × 0.2 +
  source_reliability × 0.2
```

≥ 0.9 **自动通过**
直接发布契约，无需人工介入

< 0.9 **专家审核**
转交业务专家审核低置信度子集

💡 **核心价值：**专家审核范围由"全部草稿"收敛为"未达阈值的少数"，实现人机高效协同——不是替代人，而是让人只审关键决策点。

PART THREE

落地实现细节

信任、语义契约、委派、交接、治理

03

对应模块：trust/

Agentic AICon

01

跨组织信任——Agent之间凭什么信

四层信任机制，逐层解决场景问题

场景里的问题

节点2 交接那一刻——采购方要把结算任务和付款权限交给供应商侧的某个 Agent。
它凭什么确认对方真是供应商的系统、不是仿冒的?
供应商又凭什么确认采购方 Agent 只请求了"payment_execution"、没有越权要别的?

1

组织白名单

解决"谁能连进来"
不是随便一个Agent就能连

2

mTLS证书

解决"你是你说的那个组织吗"
防冒名+防时序攻击

3

Agent Card签名

解决"你的能力声明被改过吗"
RSASSA-PSS签名验证

4

信任等级

解决"信到什么程度"
分级信任+自动降级

信任机制①：组织白名单

解决"谁能连进来"

📋 场景问题

采购方要把结算任务交给供应商侧的 Agent。
凭什么确认对方真是供应商的系统，不是仿冒的?

✅ 解决方案

不是"随便一个Agent就能连"——必须在**白名单**里。

白名单维护信任组织的核心信息：

- **certificate_fingerprint**：证书指纹
- **trust_level**：信任等级
- **allowed_capabilities**：允许的能力

trust/whitelist.py

● 简化版

```
# 信任组织白名单 TRUSTED_ORGS = { "supplier_corp": { "certificate_fingerprint": "sha256:abc123...", "trust_level": "verified", "allowed_capabilities": ["payment_execution", "goods_shipping"], }, } def is_trusted(org_id: str) -> bool: """检查组织是否在信任白名单中""" return org_id in TRUSTED_ORGS # 使用示例 if is_trusted("supplier_corp"): print("✓ 信任组织，允许连接") else: print("✗ 非信任组织，拒绝连接")
```

信任机制②：mTLS证书

解决"你是你说的那个组织吗"

📋 场景问题

即使在白名单里，如何确认连接方真的是该组织，而不是仿冒者?

✅ 解决方案

用 **mTLS 双向证书认证** + 指纹比对。

核心安全点：

- 使用 **hmac.compare_digest** 比对指纹
- 既防冒名，又防**时序攻击**

⚡ 时序攻击防护：
普通字符串比较 (==) 会因长度不同而提前返回，
攻击者可据此推测指纹内容。
hmac.compare_digest 恒定时间比较，杜绝此类攻击。

trust/certificate.py ● 简化版

```
# mTLS证书验证核心逻辑 import hmac def verify_certificate(certificate, expected_fingerprint): """验证mTLS证书 使用 hmac.compare_digest 比对指纹： – 防冒名：证书指纹必须匹配 – 防时序攻击：恒定时间比较 """ actual = compute_fingerprint(certificate) if not hmac.compare_digest(actual, expected_fingerprint): raise UntrustedOrgError( f"证书指纹不匹配: expected= {expected_fingerprint}" ) return True # 关键点：使用 hmac.compare_digest 而非 == # 防止时序攻击推断指纹内容
```

信任机制③：Agent Card签名

解决"你的能力声明被改过吗"

📋 场景问题

Agent Card 声明了能力（如 payment_execution），
如何确保这份声明在传输过程中未被篡改?

✅ 解决方案

Agent Card 使用 **RSASSA-PSS 签名**。
一旦被改，签名立刻失效。

验证流程：
1. **规范化**：json.dumps(sort_keys=True)
2. **签名验证**：RSA + PSS padding

🔐 **RSASSA-PSS 优势：**
- 比PKCS#1 v1.5更安全
- 随机化padding，防重放攻击
- salt_length=32，增强安全性

trust/signature.py ● 简化版

```
# Agent Card签名验证核心逻辑 def verify_agent_card_signature(agent_card, signature, public_key) -> bool: """验证Agent Card签名 使用 RSASSA-PSS 签名验证： - 规范化JSON（sort_keys） - PSS padding with MGF1 - SHA256哈希 """ # 1. 规范化：确保字节序一致 canonical = json.dumps(agent_card, sort_keys=True, ensure_ascii=False) try: # 2. 签名验证 public_key.verify( base64.b64decode(signature), canonical.encode(), padding.PSS( mgf=padding.MGF1(hashes.SHA256()), salt_length=32 ), hashes.SHA256(), ) return True except InvalidSignature: return False
```

信任机制④：信任等级

解决"信到什么程度"

Agentic AICon

第 4 层

📋 场景问题

即使身份验证通过，不同 Agent 应有不同信任等级，
代码变更后信任如何处理?

✅ 分级信任机制

- P **pending** – 待验证（新注册）
- U **unverified** – 未验证（代码变更后）
- V **verified** – 已验证（首次运行通过）

💡 **自动降级**：代码变更 → 自动退回 unverified
💡 **升级条件**：首次运行通过 → 升级至 verified

trust/trust_level.py

● 简化版

```
class TrustLevelManager: """信任等级管理器核心逻辑""" def upgrade_to_verified(self, agent_id, first_run_result): """升级至verified等级：首次运行结果通过""" if not first_run_result["passed"]: raise FirstRunFailedError(agent_id) self._set_trust_level(agent_id, "verified") def downgrade_on_code_change(self, agent_id): """代码变更时自动降级至unverified""" self._set_trust_level(agent_id, "unverified") # 记录降级原因 self._log_event(agent_id, "trust_downgraded", "code_changed") def _set_trust_level(self, agent_id, level): # 更新信任等级 self.store.set(f"trust:{agent_id}", level)
```

Agent Card 完整结构

标准化Agent身份与能力声明，支持跨组织发现与信任验证

结构概览

a2a（基础信息）

遵循A2A协议的标准字段

- **name** Agent名称
- **description** 功能描述
- **url** 服务地址
- **version** 版本号
- **skills** 能力列表

extensions（信任扩展）

跨组织协作的信任信息

- **agent_id** 唯一标识
- **org_id** 所属组织
- **trust_level** 信任等级
- **certificate** mTLS证书
- **signature** RSASSA-PSS签名
- **allowed_capabilities** 允许的能力

agent_card.json

简化版


```
{ "agent_card": { "a2a": { "name": "Supplier Finance Agent", "description": "Handles cross-org settlement payment execution.", "url": "https://supplier-corp.example.com/agent/finance", "version": "1.0", "skills": [ { "id": "payment_execution", "name": "Payment Execution" } ] }, "extensions": { "agent_id": "supplier_finance_agent_001", "org_id": "supplier_corp", "trust_level": "verified", "scope": "cross_org_settlement", "certificate": "-----BEGIN CERTIFICATE-----...", "signature": "sha256:signature...", "allowed_capabilities": ["payment_execution", "goods_shipping"] } } }
```


代码已简化：聚焦核心字段，完整结构见GitHub

02

对应模块：semantic/ + contracts/

Agentic AICon

语义契约——数据是什么意思、谁负责

告知层 + 推理层，解决"口径差异13万元"

场景里的问题

采购方按含税口径，供应商按不含税口径——**口径差异13万元**。
同一个词"结算金额"，双方理解不同，导致执行错误或流程阻塞。

告知层

每个字段的作用，解决"文字上就不一致"

definition
权威定义：含税/不含税、汇率口径

version
版本号：跨组织校验一致性

scope
适用范围：只在特定场景生效

owner
负责人：出问题能找到人

lineage
血缘：追溯数据来源

hash
内容寻址：防篡改

核心：差13万的根源在 definition 字段——强制双方写清楚含税/不含税、汇率口径、精度。

推理层

Ontology的作用，解决"文字一样，但推导路径不同"

class
本体类：MonetaryAmount

relations
关系——这是核心

constraints
约束：可执行化

properties
属性：currency/precision

核心：relations 把"文字一致"升级为"推导路径一致"——这才是真正解决"差13万"的机制。

语义契约核心字段

Agentic AICon

四个关键字段，逐层解决口径差异问题

definition 1

❌ 文字上就不一致

✅ 强制双方写清楚含税/不含税、汇率口径、精度

```
"definition": "Tax-inclusive total amount, based on the exchange rate on the contract signing date" // 差13万的根源： // 采购方：含税，合同签订日汇率 // 供应商：不含税，到账日汇率 // definition强制统一口径
```

version 2

❌ 文字一样，但版本不同

✅ 跨组织传递时校验版本一致性

```
"version": "v2.1" // 版本不同 → 检查双方共同支持的最近版本 // 协商降级；无交集 → 直接拒绝 // 确保双方使用同一规则版本
```

relations 3

❌ 文字一样，版本一样，但推导路径不同

✅ 强制双方对齐推导路径，取值基准内嵌在target

```
"relations": [ {"relation_type": "adjusted_by", "relation_target": "exchange_rate@contract_signing_date"} ] // 核心：都用adjusted_by: // exchange_rate@contract_signing_date // 一方签订日、一方到账日 // 字符串比对直接不等！
```

hash 4

❌ 传递过程中被篡改

✅ 内容寻址——内容变，哈希变；接收方一比对就知道

```
"hash": "sha256:99e1cd7e8fc3..." // hash覆盖整个契约内容 // 包括告知层+推理层 // 任何一处被改，hash都变
```

完整语义契约示例

Agentic AICon

settlement_amount 的完整定义，hash在顶层与term、definition平级

semantic_contract.json

● 简化版

{ "term": "settlement_amount", "definition": "Tax-inclusive total amount, based on the exchange rate on the contract signing date", "version": "v2.1", "scope": "cross_org_settlement", "owner": "procurement_finance_dept", "lineage": ["erp.purchase_order.amount", "finance.contract.amount"], "permission": "finance_agent_only", "valid_from": "2026-09-01", "ontology": { "class": "MonetaryAmount", "parent_class": "Quantity", "properties": [{"name": "currency", "type": "string"}, {"name": "precision", "type": "integer"}], "relations": [{"relation_type": "derived_from", "relation_target": "contract_amount"}, {"relation_type": "adjusted_by", "relation_target": "exchange_rate@contract_signing_date"}], "constraints": [{"constraint_type": "must_be_positive"}, {"constraint_type": "rounding", "constraint_value": "2_decimal"}] }, "hash": "sha256:99e1cd7e8fc3a6680501c7c7b802952d1863f13a4a656be7b03cedfdae5aaf0b" }

结构说明

🟢 告知层字段

- term: 术语名
- definition: 权威定义
- version: 版本号
- scope: 适用范围
- owner: 负责人
- lineage: 血缘

🟡 推理层字段 (Ontology)

- class: 本体类
- relations: 关系（核心）
- constraints: 约束
- properties: 属性

🔐 **hash 位置：**在顶层，与term、definition平级，不在ontology内部。由发布流程自动计算。

跨组织语义一致性校验

Agentic AICon

版本校验 + Ontology推导路径比对，确保双方理解一致

校验流程

1. **版本校验**
   比较remote_contract和local_contract的version字段
2. **版本协商**
   版本不同→检查共同支持的最近版本协商降级；无交集→拒绝
3. **推导路径比对**
   提取relations，比对relation_type和relation_target
4. **返回结果**
   路径完全一致→True；否则→False

💡 **核心价值：**把"文字一致"升级为"推导路径一致"——这才是真正解决"差13万"的机制。

semantic/validator.py

● 简化版

```
# 跨组织语义一致性校验核心逻辑 def validate_semantic_consistency(remote_contract, local_contract) -> bool: """版本 + Ontology推导路径比对""" # 1. 版本校验 if remote_contract["version"] != local_contract["version"]: return negotiate_version(remote_contract, local_contract) # 2. 比对Ontology推导路径 remote_relations = { (r["relation_type"], r["relation_target"]) for r in remote_contract["ontology"]["relations"] } local_relations = { (r["relation_type"], r["relation_target"]) for r in local_contract["ontology"]["relations"] } # 3. 路径比对——必须完全一致 return remote_relations == local_relations
```

💡 **代码已简化：**聚焦核心比对逻辑

对应模块：discovery/

确定性委派——交给谁、怎么选

按能力发现 + 规则化选择 + 明确兜底

场景里的问题

场景表节点2——收货确认后，采购方要把结算任务交接给供应商。但供应商侧注册了多个 Agent（发票核对、对账、付款执行），**交给哪个**？

用 LLM 猜？同一任务两次委派结果可能不同——**结算场景容不下随机性**；
按 Agent 名字硬编码？供应商换了个部署地址，流程就断。



按能力发现

不是"按名字查"——是"按能力查"，动态、灵活

- ✅ find_agents(capability, scope, trust_level)
- ✅ Consul健康检查 + passing状态



规则化选择

五条规则硬性过滤 + 排序key保证确定性

- ✅ 能力匹配 + scope匹配 + 信任达标
- ✅ 同组织优先 + URL字典序



明确兜底

四种失败场景，四种兜底策略

- ✅ 无候选→escalate
- ✅ 无验证→reject
- ✅ 无健康→retry

五条选择规则与兜底策略

把"交给谁"从随机变成确定

Agentic AICon

✅ 五条选择规则

1. **能力匹配**：task["capability"]在Agent的skills中
2. **scope匹配**：extensions.scope == task["scope"]
3. **信任达标**：extensions.trust_level == "verified"
4. **同组织优先**：org_id == task["org_id"] 排前面
5. **URL字典序**：保证确定性排序

四种兜底策略

无候选Agent	escalate
无verified Agent	reject
无健康Agent	retry
匹配成功	delegate

discovery/matcher.py

● 简化版

```
# 选择与兜底核心逻辑 def match_agent_with_fallback(task, candidates): """带兜底的匹配策略""" # 1. 检查是否有候选 if not candidates: return {"action": "escalate", "reason": "NO_CANDIDATES"} # 2. 过滤已验证的Agent verified = [a for a in candidates if a["extensions"]["trust_level"] == "verified"] if not verified: return {"action": "reject", "reason": "NO_VERIFIED_AGENT"} # 3. 过滤健康的Agent healthy = [a for a in verified if check_health(...)] if not healthy: return {"action": "retry", "reason": "NO_HEALTHY_AGENT"} # 4. 五条规则匹配 matched = match_agent(task, healthy) return {"action": "delegate", "target": matched}
```

💡 **代码已简化：聚焦核心匹配与兜底逻辑**

与LangGraph对接

匹配逻辑自研，交接机制用LangGraph原生Command(goto=...)

集成要点

匹配逻辑（自研）

- find_agents() 按能力发现
- match_agent_with_fallback() 规则+兜底
- 返回action和target

交接机制（LangGraph原生）

- Command(goto=...) 动态路由
- Command(update=...) 更新状态
- 一次调用做两件事

路由策略

delegate → 交接给供应商Agent
escalate → 转人工审批
其他 → 驳回归档

体感点：五条规则+兜底，把"交给谁"从随机变成确定。

core/nodes/procurement.py　　简化版

```
from langgraph.types import Command from typing import Literal from discovery import registry, match_agent_with_fallback def procurement_agent(state) -> Command[Literal["supplier_agent", "human_review", "reject"]]: """采购方Agent：按能力发现 + 规则选择 + 兜底策略""" # 1. 构建任务描述 task = { "capability": "payment_execution", "scope": "cross_org_settlement" } # 2. 按能力发现候选Agent candidates = registry.find_agents(capability="payment_execution") # 3. 规则匹配 + 兜底策略 result = match_agent_with_fallback(task, candidates) # 4. 根据结果路由 if result["action"] == "delegate": # 匹配成功：交接给供应商Agent return Command( update={"handoff_packet": {...}}, goto=result["target"]["agent_id"] ) elif result["action"] == "escalate": # 无候选：转人工审批 return Command( update={"error": result["reason"]}, goto="human_review" ) else: # 其他失败：驳回 return Command( update={"error": result["reason"]}, goto="reject" )
```

交接机制——HandoffPacket两层结构

Command解决"交接动作"，引用传递解决"交接内容"

场景里的问题

节点2 交接——合同、发票、口径契约带什么？全拷贝过去，**30页PDF塞爆对方上下文**；只发个订单号，**对方干不了活**。

⚡ Command：交接 + 路由

Command(goto, update) 一次调用做两件事——更新状态 + 决定下一步

HandoffPacket字段	场景里对应什么
trigger	"goods_received"（收货确认）
resource	订单号 + 合同哈希（分层存储引用）
semantic_contract	"结算金额"口径契约
evidence_refs	发票PDF哈希 + 摘要
source_agent + trace_id	采购方Agent身份 + 因果链ID

💡 **要点：** HandoffPacket不是"把状态全拷贝给对方"——是一份有清单、有引用、有责任边界的交接单。

core/nodes/procurement.py　● 简化版

```
# 构建HandoffPacket并交接 def procurement_agent(state) -> Command[Literal["supplier_agent"]]: """采购方Agent：构建HandoffPacket并交接""" # 构建HandoffPacket（核心字段） handoff_packet = { "trigger": "goods_received", # 交接触发条件 "resource": state["hash_id"], # 分层存储引用 "semantic_contract": semantic_contract, # 口径契约 "evidence_refs": [ # 证据引用 {"hash": "sha256:invoice...", "summary": "发票摘要"} ], "source_agent": "procurement_agent_001", # 源Agent "trace_id": state["trace_id"] # 因果链ID } # Command一次调用：更新状态 + 路由 return Command( update={"handoff_packet": handoff_packet}, goto="supplier_agent" )
```

💡 **代码已简化：** 聚焦核心交接逻辑

📦 引用传递机制

Agentic AICon

大文件不传内容，只传哈希ID + 摘要 + 获取工具

🎯 核心概念

🟢 LayeredStorage（存储层）

存：大文件不传内容

- <1MB → Redis缓存
- >1MB → S3对象存储
- 返回哈希ID作为引用

🟡 ReferenceLoading（加载层）

用：LLM按需获取

- 传入哈希ID列表
- 返回摘要 + fetch_artifact工具
- LLM按需调用获取完整内容

💡 **解决场景问题：**

存储层"存"——加载层"用"；

大文件不传内容，只传哈希ID + 摘要 + 获取工具；

LLM按需调用fetch_artifact，而不是一次性塞爆prompt。

handoff/storage.py

● 简化版

```
# 分层存储核心逻辑 class LayeredStorage: """分层存储：大文件存S3，小文件存Redis""" def put(self, content: bytes, ttl_hours: int = 72) -> str: """存储内容，返回哈希ID""" hash_id = "sha256:" + hashlib.sha256(content).hexdigest() # 分层存储策略 if len(content) < 1 * 1024 * 1024: # < 1MB self.redis.setex(hash_id, ttl_hours * 3600, content) else: # >= 1MB self.s3.put_object(Bucket="...", Key=hash_id, Body=content) return hash_id def get(self, hash_id: str) -> bytes: """获取内容""" content = self.redis.get(hash_id) if content: return content return self.s3.get_object(Bucket="...", Key=hash_id)["Body"].read() def verify(self, hash_id: str, content: bytes) -> bool: """验证内容完整性""" actual = "sha256:" + hashlib.sha256(content).hexdigest() return actual == hash_id # LLM按需调用的获取工具 def fetch_artifact(hash_id: str) -> str: """LLM按需调用的获取工具""" content = storage.get(hash_id) if not storage.verify(hash_id, content): raise HashMismatchError(hash_id) return content.decode("utf-8")
```

投递与存储分工

HandoffPacket经Kafka传递保证不丢不重，包内哈希引用指向的内容存于分层存储

投递层 (Kafka)

HandoffPacket经Kafka传递

- 不丢：消息持久化
- 不重：幂等消费
- 不乱序：分区保证
- 可重放：错误恢复

HandoffPacket

交接包：清单+引用+责任边界

- trigger：交接触发条件
- resource：哈希引用
- semantic_contract：口径契约
- evidence_refs：证据引用
- source_agent + trace_id

存储层 (Redis/S3)

包内哈希指向的内容

- <1MB：Redis缓存
- >1MB：S3对象存储
- 哈希验证完整性
- TTL自动清理

分工对比

维度	投递层 (Kafka)	存储层 (Redis/S3)
职责	消息传递保证	内容存储管理
解决什么	**不丢、不重、不乱序**	**大文件不塞prompt**
技术栈	Kafka + 消费者组	Redis + S3
可靠性机制	ack机制 + 幂等消费	哈希校验 + TTL
性能特点	高吞吐、低延迟	按需加载、节省内存

治理与审计——OPA同步判定

大额付款必须人工审批；越权访问必须拦截；一旦出错，必须可追溯

对应模块：governance/

场景里的问题

大额付款必须人工审批；**越权访问**必须拦截；一旦出错，必须可追溯。

⚡ OPA核心特性

默认拒绝

default allow = false，所有请求默认拒绝

✅ 显式允许

必须满足所有条件才允许：action匹配 + trust_level + scope + 非deny

越权拦截

org_id不匹配 + 未授权跨组织 → deny

检查维度

- **action**：payment_execution
- **trust_level**：verified
- **scope**：cross_org_settlement
- **org_id**：目标组织匹配
- **cross_org_authorized**：跨组织授权

governance/opa.py + policy.rego

● 简化版

```
# OPA同步判定核心逻辑 class OPAClient: def check(self, action: str, agent: dict, context: dict) -> bool: """OPA同步判定""" input_data = { "action": action, "agent": agent, "target_org_id": context.get("target_org_id"), } response = self.client.post( f"{self.opa_url}/v1/data/cross_org_settlement", json={"input": input_data}, ) result = response.json().get("result", {}) if result.get("deny"): return False return result.get("allow", False) # policy.rego – OPA策略核心 package cross_org_settlement default allow = false allow { input.action == "payment_execution" input.agent.trust_level == "verified" input.agent.scope == "cross_org_settlement" not deny } deny[reason] { input.agent.org_id != input.target_org_id not input.agent.cross_org_authorized reason := "UNAUTHORIZED_CROSS_ORG" }
```

💡 代码已简化：聚焦核心策略逻辑

影子探针——异步观察

旁路观察、1%采样、发现异常时联动OPA临时阻断

核心特性

旁路观察

不影响主流程，异步观察事件流

1%采样

随机采样1%事件，平衡性能与覆盖

异常联动

发现PII泄漏/未授权访问 → 联动OPA临时阻断

临时规则

300秒TTL，自动过期，防误杀

观察流程

- **采样观察**：random.random() < sample_rate
- **入队等待**：asyncio.Queue 异步处理
- **规则匹配**：检查是否触发告警
- **告警处理**：记录日志 + 临时阻断

governance/shadow_probe.py — 简化版

```
# 影子探针核心逻辑 import asyncio import random import time class ShadowProbe: """影子探针：旁路观察 + 异步告警""" def __init__(self, sample_rate: float = 0.01): self.sample_rate = sample_rate # 1%采样 self.queue = asyncio.Queue() self._active_temp_rules = {} self._temp_rule_ttl = 300 # 5分钟TTL async def observe(self, event: dict): """旁路观察事件""" if random.random() > self.sample_rate: return # 99%直接跳过 await self.queue.put(event) async def alert(self, alert: dict): """处理告警""" await self._log_alert(alert) # 关键异常联动OPA if alert["rule"] in ("PII_LEAK", "UNAUTHORIZED_ACCESS"): rule_key = f"{alert['event']['agent']['org_id']}->{alert['event']['target_org_id']}" # 防重复：检查是否已有临时规则 if rule_key in self._active_temp_rules: return # 添加临时阻断规则 self._active_temp_rules[rule_key] = time.time() await self._add_temporary_rule(...) print(f"临时阻断已添加: {rule_key} (TTL: 300s)")
```

代码已简化：聚焦核心观察与告警逻辑

🔗 TraceContract审计链

Agentic AICon

Annotated[list, add] Reducer——累加，不覆盖。这不是"日志"，是"证据链"

core/state.py

● 简化版

```
# TraceContract模型定义 class TraceContract(BaseModel): """TraceContract——审计链的一条记录""" trace_id: str # 同一条因果链的ID step: int # 当前步骤 principal: str # 谁触发的 component_version: str # 组件版本 policy_version: str # 策略版本 state_delta: dict # 状态变化 receipt: dict # 外部回执 timestamp: str # 时间戳 class SettlementState(BaseModel): """结算状态：audit_log自动累加""" # 关键：审计日志累加，不覆盖 audit_log: Annotated[list, add] = [] # Reducer自动累加 # 查询示例 def get_trace_by_id(state: SettlementState, trace_id: str): """查询完整因果链""" return [t for t in state.audit_log if t.trace_id == trace_id]
```

📋 字段与监管追问

字段	监管追问	怎么回答
trace_id	"完整链路是什么"	同一条因果链
step	"出问题的是哪一步"	定位到具体步骤
principal	"谁操作的"	追溯到组织+Agent
component_version	"是不是版本问题"	定位到具体组件版本
state_delta	"状态怎么变的"	还原状态变化
receipt	"外部系统真的响应了吗"	提供外部回执

可靠性保障：幂等控制与断点恢复

Agentic AICon

可靠性不是独立功能，是缠在execute_payment和core/里的约束

对应模块：reliability/

场景里的问题

节点7——付款指令因网络重试发了两次，银行扣两次款怎么办？流程中途断了从哪续？

幂等控制

同一event_id第二次直接拒绝，防止重复付款

- ✓ execute_payment执行前先查幂等表
- ✓ event_id作为幂等键
- ✓ 已处理 → 直接返回原结果
- ✓ 未处理 → 执行并记录

```
# 幂等控制核心逻辑 def execute_payment(payment_request): event_id = payment_request["event_id"] # 1. 查幂等表 if idempotency_store.exists(event_id): return idempotency_store.get(event_id) # 2. 执行付款 result = bank_api.execute(payment_request) # 3. 记录幂等 idempotency_store.set(event_id, result, ttl=86400) return result
```

代码已简化：聚焦核心幂等逻辑

断点恢复

Checkpointer持久化，故障后从最近断点续跑

- ✓ 每个节点执行后持久化状态
- ✓ interrupt()暂停时状态保存
- ✓ 故障重启后从最近checkpoint恢复
- ✓ 断电、重启、隔天再批都能续跑

```
# 断点恢复核心逻辑 # 构建图时传入checkpointer graph = build_graph(checkpointer=postgres_checkpointer) # 执行流程 config = {"configurable": {"thread_id": "settlement_001"}} result = graph.invoke(initial_state, config) # 中断后恢复执行 # 从最近checkpoint继续 resumed = graph.invoke(None, config)
```

代码已简化：聚焦核心恢复逻辑

Agentic AICon

PART FOUR

工程案例与收益

04

POC验证指标与业务收益

工程案例效果：PoC设计与预期指标

Agentic AICon

跨组织结算主链路PoC：单环境双进程模拟采购与供应商，计划执行10轮完整结算流程

PoC范围声明

- **验证范围：** 主干机制（语义契约校验、口径对齐、大额审批、幂等控制、审计链）
- **排除范围：** 多组织并发、真实银行接口对接、长期契约演化（属下一阶段生产化计划）
- **环境：** 单环境双进程模拟采购与供应商

生产就绪指标

可靠性

Kill进程/断消息队列/断缓存，各注入3次，Checkpointer均应从最近断点恢复

安全性

伪造org_id、过期证书指纹、篡改Agent Card三类攻击例，trust/+OPA预期全部拦截

成本可控

置信度评估在PoC契约集上运行，低置信度子集占比预期15–25%（生产环境需更大契约集重标定）

验证步骤与预期结果

验证步骤	验收标准	预期结果（设计目标）
1. 口径差异复现与拦截	双方按不同口径计算→校验失败→拒绝	差异稳定复现(500万 vs 487万)，semantic/校验在<100ms内拒绝并给出差异原因
2. 语义契约对齐后金额一致	双方统一v2.0契约→结果一致	双方计算结果均为500.00万，精确到分
3. 大额审批门	500万触发interrupt()→人工审批→恢复续跑	流程持久化后Kill进程，重启+审批恢复，状态无丢失
4. 幂等控制	同一event_id重放→第二次起全部拒绝	重放5次，仅首次执行，其余被幂等表拒绝并返回原回执
5. 全链路审计	每步可按trace_id回放，八问均可回答	约15步审计链逐条含principal/policy_version/state_delta/receipt

一句话： PoC将复现"双方各算各的、都显示执行成功、实际差13万"的完整冲突，并逐层验证解药——语义契约拦住口径差异，审批门拦住越权拍板，幂等表拦住重复扣款，审计链让每一步可回放。生产化是下一阶段计划。

双进程环境拓扑图：PoC运行期架构

10个模块在采购方与供应商两个进程间的分布式协作机制

采购方进程 (Procurement)

单组织编排内核

LangGraph StateGraph

对齐组 (发送端)

contracts | discovery | semantic | context

交互组 (发送端)

trust | handoff

HandoffPacket 构建器

Command(goto, update)
哈希引用 + 摘要

LayeredStorage

Redis (小文件) + S3 (大文件)
哈希ID引用存储

→

mTLS 握手

双向证书认证
hmac.compare_digest

⇄

A2A 协议交互

Agent Card 签名验证
HandoffPacket 传递

←

TraceContract 凭证

trace_id 关联
跨方对账回执

供应商进程 (Supplier)

单组织编排内核

LangGraph StateGraph

交互组 (接收端)

trust | handoff | reliability

保障组 (执行闸口)

governance | tools

OPA 默认拒绝 → 策略比对
→ 确定性代码执行

TraceContract 审计链

Annotated[list, add]
不可变累加凭证

LayeredStorage

Redis (小文件) + S3 (大文件)
fetch_artifact 按需获取

落地成果与业务收益：PoC实测与价值验证

跨组织结算主链路PoC已跑通，实测数据支撑业务价值闭环

PoC实测结果（单环境双进程）

1. 口径差异拦截（节点3）

成功复现13万差异，semantic/校验耗时**实测<50ms**，拦截率100%

2. 语义一致性对齐（节点4/5）

统一v2.0契约后，双方计算结果均为500.00万，精确到分，误差为**0**

3. 大额审批中断恢复（节点6）

500万触发interrupt()，Kill进程后重启+审批恢复，状态**零丢失**

4. 幂等控制验证（节点7）

同event_id重放5次，仅首次执行，后续4次均被幂等表拦截，回执一致

实际业务收益

效率提升

效率 跨组织对账从人工1–2天缩短至**分钟级**自动确认

风险控制

安全 重复付款风险降至**0**，越权操作100%拦截

合规成本

合规 全链路证据链，满足金融级监管“八问”，零额外审计成本

Token成本

成本 确定性代码替代概率模型执行高风险动作，Token消耗降低**90%+**

核心结论：架构已通过PoC验证，不仅解决了“口径差异”这一核心痛点，更实现了“效率、安全、合规、成本”四维度的业务价值闭环。生产化是下一阶段计划。

对比分析：现有Agent框架直连 vs 跨组织基础设施

基于产业互联网结算场景的 PoC 实测对比

评估维度	现有 Agent 框架直连 (Baseline)	跨组织基础设施架构 (Proposed)
验证成本	**消耗上万 Tokens (10k+)** 需将 30 页合同 PDF 塞入 Prompt 进行 LLM 推理	**消耗 < 50 Tokens (降低 90%+)** 通过 relations 字符串比对 (零推理成本)
静默失败率	**不可控 (事后人工对账)** Agent A 与 B 各自报成功，结果分叉无法被自动发现	**0% (拦截率 100%)** 确定性代码拦截 + 跨方凭证链对账，分歧即时暴露
跨域可靠性	**缺乏跨域断点恢复 (流程卡死)** 网络抖动或长流程中断后，状态易丢失或重复执行	**精确一次 (Exactly-Once)** event_id 幂等表 + HandoffPacket 快照 + Checkpointer 恢复
风险隔离边界	**高度耦合 (Agent 决定执行)** Agent 依据语义判断直接调用工具，幻觉直接触发不可逆动作	**纵深解耦 (分层防御)** 划分理解层(LLM)、授权层(OPA)、执行层(Tools)，幻觉无法穿透授权门控

结论：现有 Agent 框架解决了组织内的编排问题，但在跨组织场景下存在**架构缺失**。本架构并非替代现有框架，而是在其之上叠加了一层**跨组织基础设施**，通过引入确定性约束与证据链，实现了从**“尽力而为”**到**“确定性协作”**的跃迁。

威胁模型与适用域声明

明确架构防御边界，确保安全主张的信用级别

对手能力假设与攻防映射

假设	对手能力	阻断机制
A (信道)	窃听、篡改在途通信	TLS 1.3 + Agent Card RSASSA-PSS签名
B (内容)	影响Agent读取的外部内容	本架构不防 (见边界1)
C (凭证)	攻破私钥/策略存储/进程	本架构不防 (见边界3)

攻防映射表

攻击向量	阻断机制	性质
伪造Agent身份	Agent Card RSASSA-PSS签名	密码学
篡改在途任务数据	TLS + HandoffPacket哈希比对	密码学
重放已完成任务	event_id 幂等表	工程约束
伪造成果交付	交付物哈希校验+签名验证 (validate_receipt/verify_payment)	工程约束
语义歧义套利	契约definition字段比对	工程约束
伪造org_id	RSASSA-PSS签名+白名单	密码学

明示的架构边界

边界1：间接提示注入不在防御范围

本架构假定进入语义契约校验的输入已完成内容安全审查。若对手诱导一方的LLM生成了通过哈希比对的错误字段值，契约校验无效。

残余风险声明：本架构消除"未授权执行"，不消除"经授权但依据被诱导的执行"。

边界2：审计链在PoC阶段依赖进程完整性

TraceContract当前以LangGraph状态累加实现，攻破单方进程即可篡改审计链。生产化路径：哈希链+对方签名互证+外部时间戳锚定(RFC 3161)。

残余风险声明：在完成升级前，本架构主张"审计可追责"，不主张密码学意义上的不可抵赖。

边界3：双方合谋不防

审计链防单方抵赖，不防双方串通伪造一致口径。这是所有双边审计体系的设计边界。

架构失效条件

- ❌ 任一方私钥泄露且未在信任等级降级前被发现；
- ❌ 对手找到绕过Agent Card校验的通信路径（如直连Redis）；
- ❌ 契约definition字段双方使用同一污染语料生成；
- ❌ OPA策略文件被单方篡改且未纳入审计。

💡 **核心立场：**分层架构的定义是每一层只对自己层的威胁负责，这是适用域声明，而非缺陷声明。

PART FIVE

复盘思考与展望

05

Agent Card作为基础设施的未来

Agent Card：多Agent管理的基础设施

Agentic AICon

标准化身份与能力声明，支撑跨组织Agent生态的未来演进

战略价值

标准化身份：统一的Agent身份格式，支持跨组织发现与验证

能力声明：skills字段声明Agent能力，支持**按能力发现**而非按名字查找

信任传递：extensions字段携带信任信息，构建**跨组织信任链**

版本管理：version字段支持Agent迭代，代码变更自动触发信任降级

生态互联：遵循A2A协议，为未来**Agent互联网**奠定基础

基础设施能力

服务发现

基于能力而非名字的动态发现

信任验证

四层信任机制的身份基础

能力治理

allowed_capabilities权限控制

版本演进

支持Agent迭代与兼容性管理

未来展望：Agent Card是构建Agent互联网的基石——当每个Agent都有标准化身份和能力声明，跨组织协作将像API调用一样简单。

Agent Card：演进路径与未来展望

从当前实现到Agent互联网的三阶段演进

1

当前实现

跨组织结算场景的基础设施

- 标准化Agent身份与能力声明
- 四层信任机制的身份基础
- 按能力发现与服务注册
- 版本管理与信任降级
- 跨组织语义契约传递

2

近期演进

多场景Agent协作平台

- Agent能力市场与交易平台
- 动态信任评估与信誉系统
- 跨组织Agent工作流编排
- Agent能力组合与编排
- 标准化API与协议扩展

3

未来愿景

Agent互联网生态

- Agent互联网协议标准化
- 跨行业Agent互联互通
- Agent自主发现与协作
- 去中心化Agent信任网络
- Agent经济与价值流转

核心洞察：Agent Card不仅是当前跨组织协作的基础设施，更是未来Agent互联网的基石。正如TCP/IP协议之于互联网，Agent Card之于Agent互联网——标准化身份与能力声明是构建下一代AI协作生态的关键。

PART SIX

Takeaway总结

06

听众视角的核心观点与收益

Take away 总结参考（听众视角）

Agentic AICon

1. Agent Card是跨组织协作信任的载体 -- “白名单+mTLS+签名+信任等级”四层设防都验它，它标准化了“我是谁，能干什么，信到什么程度”
2. 口径差13万的解药不是prompt调优 -- 是基于Ontology的语义契约+推导路径比对
3. exchange_rate@签订日 内嵌relation_target -- 字符串比对拦截，零推理成本
4. “交给谁”不许LLM猜，基于Agent Card的skills声明 -- 五条规则+四种兜底策略，委派必须确定
5. 大文件只传哈希+摘要 -- LLM按需fetch, 30 页合同 + 20 张发票零塞爆
6. 上下文两层分工 -- 上下文治理（资产层）管“语义契约怎么来”+上下文管理（运行时层）管“每次调用LLM给多少，怎么给”
7. 副作用动作全是确定性代码 -- LLM只做理解和判断，风险越高越不用概率模型
8. 审计链是证据链不是日志 -- TraceContract构建因果链，让协作不可抵赖，责任可追溯，监管八问皆可答

没有 语义契约与信任层，跨组织Agent协作不是自动化，是在自动化地出错

AgenticAICon

智能体应用与架构工程大会

感谢聆听



关注公众号



更多会议详情